الجامعة العربية الأمريكية
ARAB AMERICAN UNIVERSITY

FACULTY OF ENGINEERING

COMPUTER SYSTEMS ENGINEERING DEPARTMENT

Introduction To Database Systems

Project: ICON CINEMA

MR. Hussein Younis

Group Members:

Bahaa Aldeen Abdullah	202210375
Fares Khader	202210561
Ibraheem Awad	202210491

INTRODUCTION:

ICON CINEMA requires a comprehensive and well-structured database to efficiently manage its daily operations and provide exceptional customer experience. This database is designed to streamline the cinema's processes by integrating critical data across multiple entities such as customers, movies, employees, departments, tickets, payments, and more.

The database includes features to organize customer information, manage movie schedules, allocate seats, process payments, and sell snacks. Departments play a pivotal role in overseeing key operations, ensuring smooth coordination across halls, showtimes, ticket sales, payment processing, and snack management. The relationships between employees, departments, and other entities reflect the structured hierarchy and responsibilities within the cinema.

This project establishes a robust foundation for ICON CINEMA to enhance its operational efficiency, support decision-making, and adapt to future business needs. By leveraging this database, ICON CINEMA can maintain a seamless and enjoyable experience for its customers while achieving operational excellence.

DATA REQUIREMENT:

Each cinema operates under a central management structure, with various departments and employees working collaboratively to provide exceptional movie experience for customers.

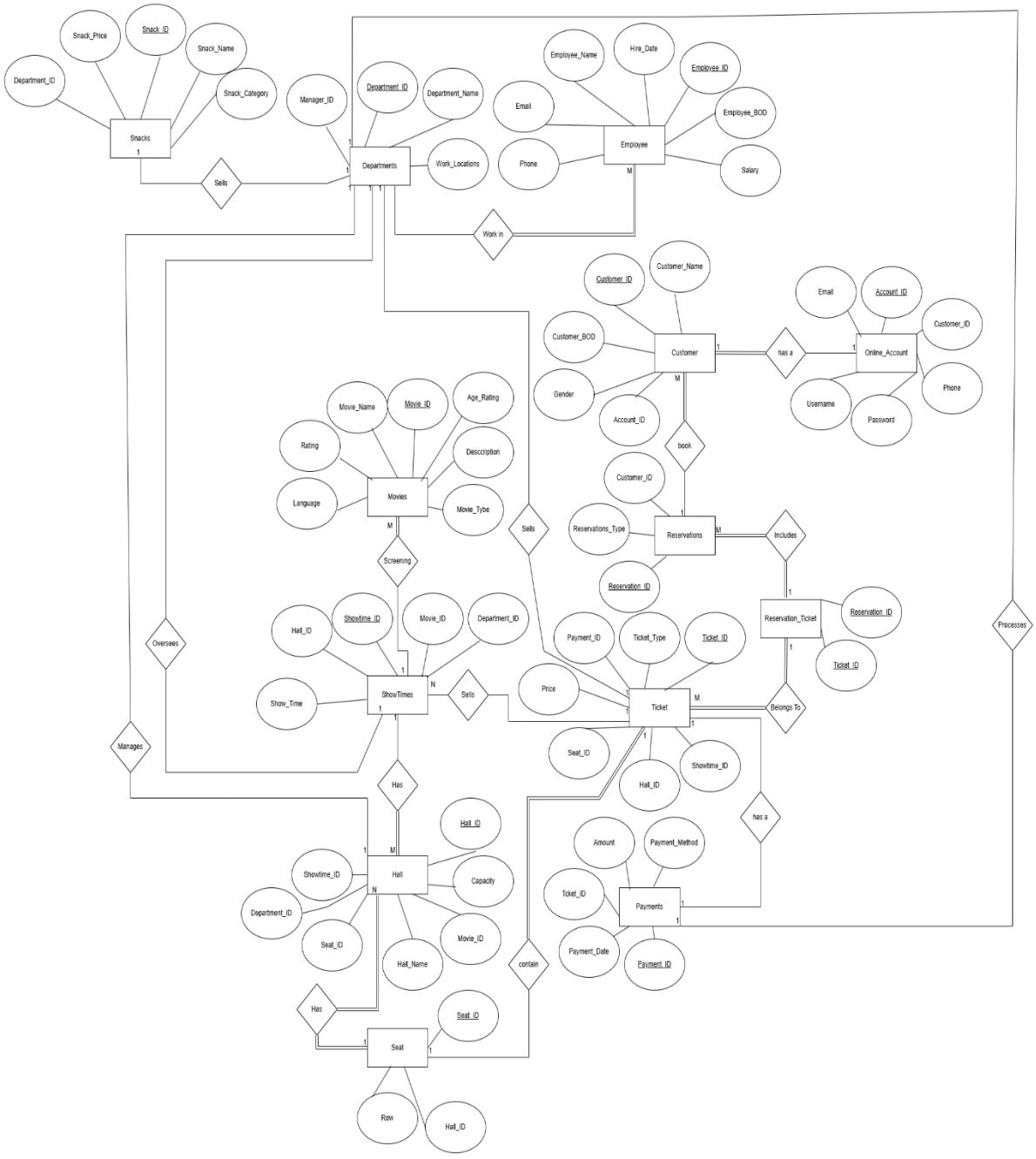
ICON CINEMA includes multiple halls, each managed by a department and overseen by employees. Halls are integral to hosting movie showtimes and are identified by unique hall IDs, with details like capacity and cinema address. Departments supervise operations such as showtime scheduling, ticket sales, payment processing, and snack management.

Customers are the core of the system, with each customer having unique personal details like name, DOB, gender, email, and phone number. Customers can make reservations (online or in-person) for movie tickets, which are linked to specific showtimes and seats. Every reservation is tied to a customer ID, and tickets are uniquely identified by ticket IDs, which also detail ticket type and price. Payments for tickets are processed by employees using various methods like cash or credit.

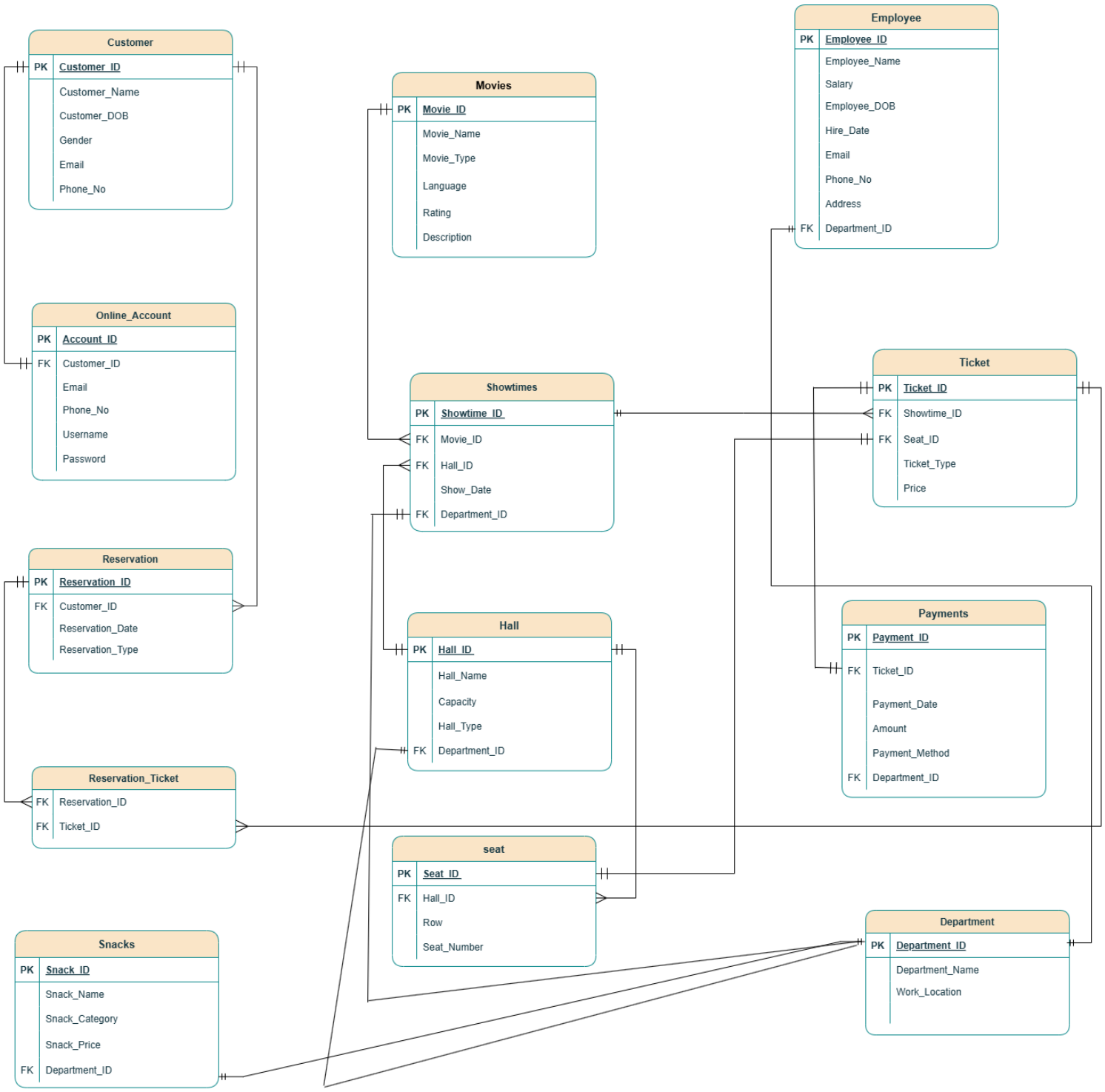
The cinema also offers a variety of snacks, categorized by type, with prices set by the relevant department. Employees are responsible for selling snacks and tickets and managing customer interactions.

The database ensures seamless coordination between customers, employees, showtimes, halls, and departments, with robust mechanisms for seat allocation, payment tracking, and operational oversight. This interconnected structure enables ICON CINEMA to deliver an efficient and memorable experience for all its patrons.

EER DIAGRAM:


Snack_Price
Snack_ID
Snack_Name
Department_ID
Snack_Category
Snacks
Sells
Manager_ID
Department_ID
Department_Name
Departments
Work_Locations
Employee_Name
Hire_Date
Employee_ID
Email
Employee_BOD
Employee
Phone
Salary
Work in
Customer_ID
Customer_Name
Customer_BOD
Customer
has a
Email
Account_ID
Customer_ID
Online_Account
Gender
Account_ID
Username
Password
Phone
book
Movie_Name
Movie_ID
Age_Rating
Rating
Desccription
Movies
Language
Movie_Type
Screening
Customer_ID
Reservations_Type
Reservations
Includes
Sells
Reservation_ID
Reservation_Ticket
Reservation_ID
Ticket_ID
Hall_ID
Showtime_ID
Movie_ID
Department_ID
Payment_ID
Ticket_Type
Ticket_ID
Oversees
Processes
Show_Time
ShowTimes
Sells
Price
Ticket
Belongs To
Manages
Seat_ID
Hall_ID
Showtime_ID
Has
has a
Hall_ID
Amount
Payment_Method
Showtime_ID
Hall
Capacity
Ticket_ID
Payments
Department_ID
Seat_ID
Movie_ID
Payment_Date
Payment_ID
Hall_Name
contain
Has
Seat_ID
Seat
Row
Hall_ID

SCHEMA:


Customer
PK Customer_ID
Customer_Name
Customer_DOB
Gender
Email
Phone_No
Movies
PK Movie_ID
Movie_Name
Movie_Type
Language
Rating
Description
Employee
PK Employee_ID
Employee_Name
Salary
Employee_DOB
Hire_Date
Email
Phone_No
Address
FK Department_ID
Online_Account
PK Account_ID
FK Customer_ID
Email
Phone_No
Username
Password
Showtimes
PK Showtime_ID
FK Movie_ID
FK Hall_ID
Show_Date
FK Department_ID
Ticket
PK Ticket_ID
FK Showtime_ID
FK Seat_ID
Ticket_Type
Price
Reservation
PK Reservation_ID
FK Customer_ID
Reservation_Date
Reservation_Type
Hall
PK Hall_ID
Hall_Name
Capacity
Hall_Type
FK Department_ID
Payments
PK Payment_ID
FK Ticket_ID
Payment_Date
Amount
Payment_Method
FK Department_ID
Reservation_Ticket
FK Reservation_ID
FK Ticket_ID
seat
PK Seat_ID
FK Hall_ID
Row
Seat_Number
Snacks
PK Snack_ID
Snack_Name
Snack_Category
Snack_Price
FK Department_ID
Department
PK Department_ID
Department_Name
Work_Location

NORMALIZATION:

1. **First Normal Form (1NF):**

- Each column contains atomic (indivisible) values.
- Each row is unique.
- There are no repeating groups.

Observations:

The table already satisfies 1NF because:

Each column contains atomic values.

Each row is unique

There are no repeating groups (we avoid it by using Reservation_Ticket table for many to many relation)

2. **Second Normal Form (2NF)**

- It is in 1NF.
- All non-prime attributes are fully functionally dependent on the entire primary key (no partial dependencies).

Observations:

All tables don't have partial dependencies and are functionally dependent on the entire primary key

Reservation_Ticket table is pure junction table with no partial dependencies**.**

3. **Third Normal Form (3NF)**

- It is in 2NF.
- No transitive dependencies (non-prime attributes must depend only on the primary key, not on other non-prime attributes).

Observations:

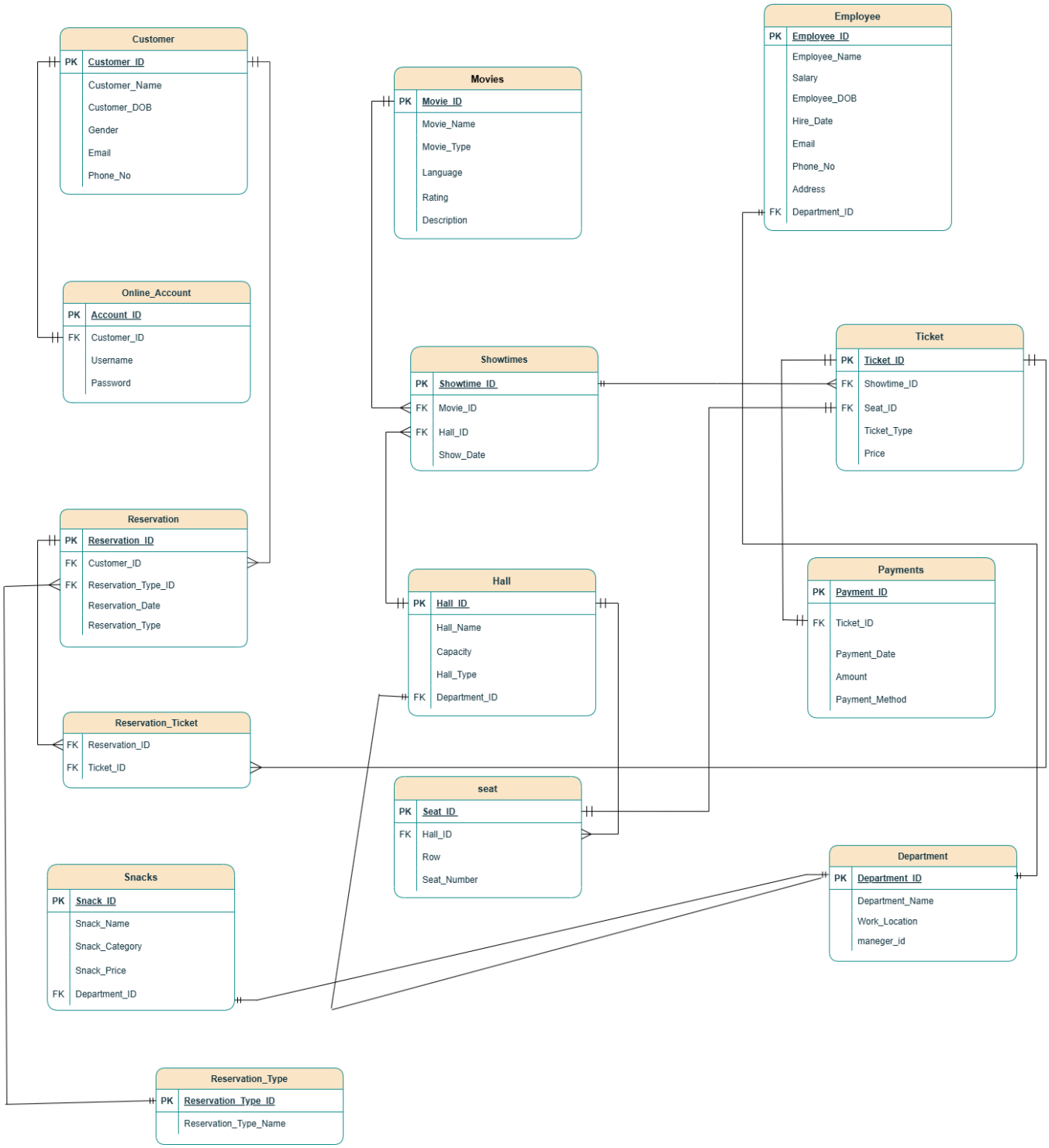
Each non-prime attribute should depend only on the primary key.
No attribute should depend on another non-prime attribute (transitive dependency).
All tables are normalized in 3 types of normalization (!NF,2NF,3NF)

Third Normal Form (3NF)

Most tables are in 3NF, but there are a few areas where transitive dependencies or redundancies exist:

- **Online_Account:** Table Attributes(Email)and(Phone_No) are duplicated from the Customer table. These attributes are dependent on Customer_ID, not the primary key (Account_ID), which violates 3NF.
- **Showtimes Table:** The Department_ID attribute is redundant because it can be derived through the Hall_ID foreign key (from the Hall table).
- **(Payments Table)** The Department_ID attribute is redundant because it can be derived through the Ticket_ID foreign key (from the Ticket table, which links to Showtimes, which links to Hall, which links to Department).
- Reservation_Type in the Reservation table has a limited set of values ("Online", "Offline"),we need to create a separate lookup table

SHCEMA AFTER NORMALIZATION:



Functional Dependencies:

Customer Table:

Cutomer_ID → Customer_Name,Customer_DOB,Gnder,Email,Phone_No

Department Table:

Department_ID → Department_Name,Work_Location,Manager_ID

Movies Table:

Movie_ID →
Movie_Name,Movie_Type,Language_Of_Movie,Rating,Descreption_for_movie

Employee Table:

Employee_ID
→Employee_Name,Salary,Employee_DOB,Email,hire_date,Phone_No,Address_of_employee,

departmet_id

Accunt Online Table :

Account_ID → Customer_ID,Username,Passwords

Hall Table :

Hall_ID → Hall_Name,Capacity,Hall_Type,Department_ID

Show Times Table :

Showtimes_ID → Movie_ID,Hall_ID,Show_Time

Seat Table :

Seat_ID → Hall_ID,Row_Num,Seat_Number

Ticket Table :

Ticket_ID → Showtimes_ID,Seat_ID,Ticket_Type,Price

Reservation Table :

Reservation_ID → Customer_ID,Reservation_Type_ID,Reservation_Type

Reservation Type Table :

Rservation_Type_ID → Reservation_Type

Reservation Ticket Table : (Intermediate Table)

(Rservation_ID,Ticket_Id) → Rservation_ID,Ticket_ID

Payments Table :

Payment_ID → Ticket_ID,Payment_Date,Amount,Payment_Method

Snacks Table :

Snack_ID → Snack_Name,Snack_Category,Snack_Price,Depertmant_ID

Tables creation:

```
CREATE TABLE Customer (
   Customer_ID NUMBER GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
   Customer_Name VARCHAR2(70) NOT NULL,
   Customer_DOB DATE,
   Gender CHAR(1),--male or female
   Email VARCHAR2(100) NOT NULL,
   Phone_No VARCHAR2(15) NOT NULL
);

CREATE TABLE Reservation_Type (
   Reservation_Type_ID NUMBER GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
   Reservation_Type_Name VARCHAR2(50) NOT NULL
);

CREATE TABLE Department (
   Dep_ID NUMBER GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
   Department_Name VARCHAR2(100) NOT NULL,
   Work_Location VARCHAR2(200) NOT NULL,
   Maneger_Id NUMBER
);

CREATE TABLE Movies (
   Movie_ID NUMBER GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
   Movie_Name VARCHAR2(100) NOT NULL,
   Movie_Type VARCHAR2(50),
   Language_of_movie VARCHAR2(50),
   Rating NUMBER(3, 1),
   Description_for_movie VARCHAR2(500)
```

```
);

CREATE TABLE Employee (
   Employee_ID NUMBER GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
   Employee_Name VARCHAR2(100) NOT NULL,
   Salary NUMBER(6, 2) NOT NULL,
   Employee_DOB DATE,
   Hire_Date DATE NOT NULL,
   Email VARCHAR2(100) NOT NULL,
   Phone_No VARCHAR2(15) NOT NULL,
   Address_of_Employee VARCHAR2(200) NOT NULL,
   Dep_ID NUMBER,
   CONSTRAINT fk_employee_department FOREIGN KEY (Dep_ID) REFERENCES Department(Dep_ID)
);

CREATE TABLE Online_Account (
   Account_ID NUMBER GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
   Customer_ID NUMBER NOT NULL,
   Username VARCHAR2(50) NOT NULL,
   Passwords VARCHAR2(50) NOT NULL,
   CONSTRAINT fk_online_account_customer FOREIGN KEY (Customer_ID) REFERENCES Customer(Customer_ID)
);

CREATE TABLE Hall (
   Hall_ID NUMBER GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
   Hall_Name VARCHAR2(100) NOT NULL,
   Capacity NUMBER NOT NULL,
   Hall_Type VARCHAR2(50) NOT NULL,
```

```
    Dep_ID NUMBER,
    CONSTRAINT fk_hall_department FOREIGN KEY (Dep_ID) REFERENCES
Department(Dep_ID)
);

CREATE TABLE Showtimes (
    Showtime_ID NUMBER GENERATED BY DEFAULT AS IDENTITY PRIMARY
KEY,
    Movie_ID NUMBER NOT NULL,
    Hall_ID NUMBER NOT NULL,
    Show_Date DATE NOT NULL,
    CONSTRAINT fk_showtimes_movie FOREIGN KEY (Movie_ID) REFERENCES
Movies(Movie_ID),
    CONSTRAINT fk_showtimes_hall FOREIGN KEY (Hall_ID) REFERENCES
Hall(Hall_ID)
);


CREATE TABLE Seat (
    Seat_ID NUMBER GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    Hall_ID NUMBER NOT NULL,
    Row_num VARCHAR2(10) NOT NULL,
    Seat_Number NUMBER NOT NULL,
    CONSTRAINT fk_seat_hall FOREIGN KEY (Hall_ID) REFERENCES Hall(Hall_ID)
);


CREATE TABLE Ticket (
    Ticket_ID NUMBER GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    Showtime_ID NUMBER NOT NULL,
    Seat_ID NUMBER NOT NULL,
    Ticket_Type VARCHAR2(50) NOT NULL,
    Price NUMBER(6, 2) NOT NULL,
```

```
    CONSTRAINT fk_ticket_showtime FOREIGN KEY (Showtime_ID) REFERENCES Showtimes(Showtime_ID),
    CONSTRAINT fk_ticket_seat FOREIGN KEY (Seat_ID) REFERENCES Seat(Seat_ID)
);

CREATE TABLE Reservation (
    Reservation_ID NUMBER GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    Customer_ID NUMBER NOT NULL,
    Reservation_Type_ID NUMBER NOT NULL,
    Reservation_Date DATE NOT NULL,
    CONSTRAINT fk_reservation_customer FOREIGN KEY (Customer_ID) REFERENCES Customer(Customer_ID),
    CONSTRAINT fk_reservation_type FOREIGN KEY (Reservation_Type_ID) REFERENCES Reservation_Type(Reservation_Type_ID)
);


CREATE TABLE Reservation_Ticket (
    Reservation_ID NUMBER NOT NULL,
    Ticket_ID NUMBER NOT NULL,
    PRIMARY KEY (Reservation_ID, Ticket_ID),
    CONSTRAINT fk_reservation_ticket_reservation FOREIGN KEY (Reservation_ID) REFERENCES Reservation(Reservation_ID),
    CONSTRAINT fk_reservation_ticket_ticket FOREIGN KEY (Ticket_ID) REFERENCES Ticket(Ticket_ID)
);

CREATE TABLE Payments (
    Payment_ID NUMBER GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    Ticket_ID NUMBER NOT NULL,
    Payment_Date DATE NOT NULL,
```

```
    Amount NUMBER(6, 2) NOT NULL,
    Payment_Method VARCHAR2(50) NOT NULL,
    CONSTRAINT fk_payments_ticket FOREIGN KEY (Ticket_ID) REFERENCES
Ticket(Ticket_ID)
);

CREATE TABLE Snacks (
    Snack_ID NUMBER GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    Snack_Name VARCHAR2(100) NOT NULL,
    Snack_Category VARCHAR2(50) NOT NULL,
    Snack_Price NUMBER(6, 2) NOT NULL,
    Dep_ID NUMBER,
    CONSTRAINT fk_snacks_department FOREIGN KEY (Dep_ID) REFERENCES
Department(Dep_ID)
);
```

Insertion Data into Tables:

```
INSERT INTO CUSTOMER
(CUSTOMER_ID,CUSTOMER_NAME,Customer_DOB,GENDER,EMAIL,PHONE_NO) VALUES

(1,'Bahaa AbuSalameh',TO_DATE('2004-10-11', 'YYYY-MM-DD'),'M','BahaaAbuSalameh@gmail.com','0598612372'),

(2,'Basel Salah',TO_DATE('2004-5-13', 'YYYY-MM-DD'),'M','BaselSalah@gmail.com','0593963781'),

(3,'Ibrahim Awad',TO_DATE('2004-5-5', 'YYYY-MM-DD'),'M','IbrahimAwad@gmail.com','0592390041'),

(4,'Fares Khader',TO_DATE('2004-7-23', 'YYYY-MM-DD'),'M','FaresKhader@gmail.com','0597702513'),

(5,'Nihad Malli',TO_DATE('2004-3-2', 'YYYY-MM-DD'),'M','NihadMallli@gmail.com','0594398464'),

(6,'Ahmad Swafta',TO_DATE('2004-7-30', 'YYYY-MM-DD'),'M','AhmadSwafta@gmail','0594582155'),

(7,'Ameer Raid',TO_DATE('2005-2-4', 'YYYY-MM-DD'),'M','AmeerRaid@gmail.com','0594929137'),

(8,'Hadi Kmail',TO_DATE('2004-11-21', 'YYYY-MM-DD'),'M','HadiKmail@gmail.com','0569365860'),

(9,'Akram Zakarna',TO_DATE('2004-3-22', 'YYYY-MM-DD'),'M','AkramZakarna@gmail.com','0594508121'),

(10,'Zaid Ardah',TO_DATE('2005-1-15', 'YYYY-MM-DD'),'M','ZaidArdah@gmail.com','0594395564');


INSERT INTO Reservation_Type (Reservation_Type_Name)

VALUES

('Online'),

('In-Person');


INSERT INTO Department (Dep_ID, Department_Name, Work_Location, Maneger_Id)

VALUES

(1, 'Halls', 'Main Cinema Hall', NULL),

(2, 'VIP Lounge', 'Cinema Complex, VIP Area', NULL),
```

(3, 'Snack Bar', 'Main Cinema Hall, Concessions Area', NULL);

INSERT INTO Movies (Movie_Name, Movie_Type, Language_of_movie, Rating, Description_for_movie)

VALUES

('The Blue Elephant', 'Thriller', 'Arabic', 8.5, 'A psychological thriller about a psychiatrist and his mysterious patient.'),

('The Yacoubian Building', 'Drama', 'Arabic', 8.2, 'A story about the lives of residents in a historic Cairo building.'),

('Clash', 'Action', 'Arabic', 7.9, 'A political drama set in the aftermath of the Egyptian revolution.'),

('Sheikh Jackson', 'Drama', 'Arabic', 7.7, 'A story about a preacher who struggles with his love for Michael Jackson.'),

('The Night of Counting the Years', 'Historical', 'Arabic', 8.8, 'A classic film about tomb robbers in ancient Egypt.'),

('Asmaa', 'Drama', 'Arabic', 8.1, 'A story about a woman living with HIV in Egypt.'),

('The Island', 'Comedy', 'Arabic', 7.5, 'A comedy about a group of people stranded on an island.'),

('The Gate of Sun', 'Drama', 'Arabic', 7.8, 'A story about Palestinian refugees in Lebanon.'),

('The Journey', 'Adventure', 'Arabic', 8.0, 'An epic tale of a merchant’s journey across the desert.'),

('The Square', 'Documentary', 'Arabic', 8.4, 'A documentary about the Egyptian revolution.'),

('The Shawshank Redemption', 'Drama', 'English', 9.3, 'Two imprisoned men bond over several years, finding solace and eventual redemption through acts of common decency.'),

('The Godfather', 'Crime', 'English', 9.2, 'The aging patriarch of an organized crime dynasty transfers control of his clandestine empire to his reluctant son.'),

('The Dark Knight', 'Action', 'English', 9.0, 'When the menace known as the Joker emerges, Batman must confront chaos and disorder in Gotham City.'),

('John Wick', 'Action', 'English', 8.2, 'An ex-hitman comes out of retirement to track down the gangsters who killed his dog and stole his car.'),

('Inception', 'Sci-Fi', 'English', 8.8, 'A thief who steals corporate secrets through dream-sharing technology is given a final chance to have his criminal record erased.'),

('The Lord of the Rings: The Return of the King', 'Adventure', 'English', 9.0, 'Gandalf and Aragorn lead the World of Men against Sauron’s army to draw his gaze from Frodo and Sam as they approach Mount Doom.'),

('Pulp Fiction', 'Crime', 'English', 8.9, 'The lives of two mob hitmen, a boxer, a gangster’s wife, and a pair of diner bandits intertwine in four tales of violence and redemption.'),

('Forrest Gump', 'Drama', 'English', 8.8, 'The presidencies of Kennedy and Johnson, the events of Vietnam, Watergate, and other history unfold through the perspective of an Alabama man with an IQ of 75.'),

('The Matrix', 'Sci-Fi', 'English', 8.7, 'A computer hacker learns about the true nature of reality and his role in the war against its controllers.'),

('Gladiator', 'Action', 'English', 8.5, 'A former Roman General sets out to exact vengeance against the corrupt emperor who murdered his family and sent him into slavery.');

INSERT INTO Employee (Employee_Name, Salary, Employee_DOB, Hire_Date, Email, Phone_No, Address_of_Employee, Dep_ID)

VALUES

('Ahmed Samy', 8000.00, TO_DATE('1985-03-10', 'YYYY-MM-DD'), TO_DATE('2010-05-15', 'YYYY-MM-DD'), 'ahmed.samy@gmail.com', '0591122334', '123 Main St, Ramallah', 1),

('Mona Adel', 7500.00, TO_DATE('1988-07-22', 'YYYY-MM-DD'), TO_DATE('2012-08-20', 'YYYY-MM-DD'), 'mona.adel@gmail.com', '0592233445', '456 Olive St, Nablus', 2),

('Karim Mohamed', 9000.00, TO_DATE('1990-11-05', 'YYYY-MM-DD'), TO_DATE('2015-02-10', 'YYYY-MM-DD'), 'karim.mohamed@gmail.com', '0593344556', '789 Mountain St, Hebron', 3),

('Nadia Hassan', 8500.00, TO_DATE('1987-09-18', 'YYYY-MM-DD'), TO_DATE('2014-06-25', 'YYYY-MM-DD'), 'nadia.hassan@gmail.com', '0595566677', '321 Star St, Bethlehem', 1),

('Yasser Ahmed', 8200.00, TO_DATE('1992-04-12', 'YYYY-MM-DD'), TO_DATE('2018-03-08', 'YYYY-MM-DD'), 'yasser.ahmed@gmail.com', '0596677889', '654 Garden St, Jenin', 2),

('Lina Mahmoud', 7800.00, TO_DATE('1995-08-30', 'YYYY-MM-DD'), TO_DATE('2019-07-12', 'YYYY-MM-DD'), 'lina.mahmoud@gmail.com', '0597788990', '987 Peace St, Tulkarm', 3),

('Omar Salah', 8700.00, TO_DATE('1989-12-15', 'YYYY-MM-DD'), TO_DATE('2016-11-20', 'YYYY-MM-DD'), 'omar.salah@gmail.com', '0598899001', '654 Hope St, Qalqilya', 1),

('Rana Khaled', 8300.00, TO_DATE('1993-06-25', 'YYYY-MM-DD'), TO_DATE('2020-09-05', 'YYYY-MM-DD'), 'rana.khaled@gmail.com', '0599900112', '321 Palm St, Jericho', 2),

('Ali Ibrahim', 9100.00, TO_DATE('1986-02-18', 'YYYY-MM-DD'), TO_DATE('2013-04-10', 'YYYY-MM-DD'), 'ali.ibrahim@gmail.com', '0590011223', '123 Valley St, Tubas', 3),

('Hana Samir', 7900.00, TO_DATE('1998-03-08', 'YYYY-MM-DD'), TO_DATE('2021-01-22', 'YYYY-MM-DD'), 'hana.samir@gmail.com', '0591122334', '456 Sunrise St, Salfit',1);

INSERT INTO Online_Account (Customer_ID, Username, Passwords)

VALUES

(1, 'BahaaAbuSalameh', 'Bahaa@2004'),

(2, 'BaselSalah', 'Basel@2004'),

(3, 'IbrahimAwad', 'Ibrahim@2004'),

(4, 'FaresKhader', 'Fares@2004'),

(5, 'NihadMalli', 'Nihad@2004'),

(6, 'AhmadSwafta', 'Ahmad@2004'),

(7, 'AmeerRaid', 'Ameer@2005'),

(8, 'HadiKmail', 'Hadi@2004'),

(9, 'AkramZakarna', 'Akram@2004'),

(10, 'ZaidArdah', 'Zaid@2005');

INSERT INTO Hall (Hall_Name, Capacity, Hall_Type, Dep_ID)

VALUES

('Main Theater', 200, 'IMAX', 1),

('VIP Hall', 50, 'VIP', 2),

('Standard Hall A', 100, 'Standard', 3),

('Standard Hall B', 100, 'Standard', 1),

('3D Hall', 150, '3D', 2);

```
INSERT INTO Showtimes (Movie_ID, Hall_ID, Show_Date)
VALUES
(1, 1, TO_DATE('2025-01-24 06:00 PM', 'YYYY-MM-DD HH:MI AM')),
(2, 2, TO_DATE('2025-01-24 08:00 PM', 'YYYY-MM-DD HH:MI AM')),
(3, 3, TO_DATE('2025-01-24 10:00 PM', 'YYYY-MM-DD HH:MI AM')),
(4, 1, TO_DATE('2025-01-24 07:00 PM', 'YYYY-MM-DD HH:MI AM')),
(5, 2, TO_DATE('2025-01-24 09:00 PM', 'YYYY-MM-DD HH:MI AM')),
(6, 1, TO_DATE('2025-01-25 05:00 PM', 'YYYY-MM-DD HH:MI AM')),
(7, 2, TO_DATE('2025-01-25 07:00 PM', 'YYYY-MM-DD HH:MI AM')),
(8, 3, TO_DATE('2025-01-25 09:00 PM', 'YYYY-MM-DD HH:MI AM')),
(9, 1, TO_DATE('2025-01-25 06:00 PM', 'YYYY-MM-DD HH:MI AM')),
(10, 2, TO_DATE('2025-01-25 08:00 PM', 'YYYY-MM-DD HH:MI AM')),
(1, 3, TO_DATE('2025-01-26 04:00 PM', 'YYYY-MM-DD HH:MI AM')),
(2, 1, TO_DATE('2025-01-26 06:00 PM', 'YYYY-MM-DD HH:MI AM')),
(3, 2, TO_DATE('2025-01-26 08:00 PM', 'YYYY-MM-DD HH:MI AM')),
(4, 3, TO_DATE('2025-01-26 07:00 PM', 'YYYY-MM-DD HH:MI AM')),
(5, 1, TO_DATE('2025-01-26 09:00 PM', 'YYYY-MM-DD HH:MI AM'));
```

```
INSERT INTO Seat (Hall_ID, Row_num, Seat_Number)
VALUES
(1, 'A', 1),
(1, 'A', 2),
(1, 'A', 3),
(1, 'B', 1),
(1, 'B', 2),
(1, 'B', 3),
```

(2, 'A', 1),
(2, 'A', 2),
(2, 'A', 3),
(2, 'B', 1),
(2, 'B', 2),
(2, 'B', 3),
(3, 'A', 1),
(3, 'A', 2),
(3, 'A', 3),
(3, 'B', 1),
(3, 'B', 2),
(3, 'B', 3),
(4, 'A', 1),
(4, 'A', 2),
(4, 'A', 3),
(4, 'B', 1),
(4, 'B', 2),
(4, 'B', 3),
(5, 'A', 1),
(5, 'A', 2),
(5, 'A', 3),
(5, 'B', 1),
(5, 'B', 2),
(5, 'B', 3);

INSERT INTO Ticket (Showtime_ID, Seat_ID, Ticket_Type, Price)
VALUES
(1, 1, 'Regular', 50.00),
(1, 2, 'VIP', 100.00),
(2, 3, 'Regular', 55.00),

(2, 4, 'VIP', 110.00),
(3, 5, 'Regular', 60.00),
(3, 6, 'VIP', 120.00),
(4, 7, 'Regular', 65.00),
(4, 8, 'VIP', 130.00),
(5, 9, 'Regular', 70.00),
(5, 10, 'VIP', 140.00),
(6, 11, 'Regular', 75.00),
(6, 12, 'VIP', 150.00),
(7, 13, 'Regular', 80.00),
(7, 14, 'VIP', 160.00),
(8, 15, 'Regular', 85.00),
(8, 16, 'VIP', 170.00),
(9, 17, 'Regular', 90.00),
(9, 18, 'VIP', 180.00),
(10, 19, 'Regular', 95.00),
(10, 20, 'VIP', 190.00);

INSERT INTO Reservation (Customer_ID, Reservation_Type_ID, Reservation_Date)
VALUES
(1, 1, TO_DATE('2025-01-24', 'YYYY-MM-DD')),
(2, 1, TO_DATE('2025-01-24', 'YYYY-MM-DD')),
(3, 2, TO_DATE('2025-01-25', 'YYYY-MM-DD')),
(4, 2, TO_DATE('2025-01-25', 'YYYY-MM-DD')),
(5, 1, TO_DATE('2025-01-26', 'YYYY-MM-DD')),
(6, 1, TO_DATE('2025-01-26', 'YYYY-MM-DD')),
(7, 2, TO_DATE('2025-01-24', 'YYYY-MM-DD')),
(8, 2, TO_DATE('2025-01-24', 'YYYY-MM-DD')),
(9, 1, TO_DATE('2025-01-25', 'YYYY-MM-DD')),

```
(10, 1, TO_DATE('2025-01-25', 'YYYY-MM-DD'));
```

```
INSERT INTO Reservation_Ticket (Reservation_ID, Ticket_ID)
VALUES
(1, 1),
(2, 2),
(3, 3),
(4, 4),
(5, 5),
(6, 6),
(7, 7),
(8, 8),
(9, 9),
(10, 10);
```

```
INSERT INTO Payments (Ticket_ID, Payment_Date, Amount, Payment_Method)
VALUES
(1, TO_DATE('2025-01-24', 'YYYY-MM-DD'), 50.00, 'Credit Card'),
(2, TO_DATE('2025-01-24', 'YYYY-MM-DD'), 100.00, 'Debit Card'),
(3, TO_DATE('2025-01-24', 'YYYY-MM-DD'), 55.00, 'Cash'),
(4, TO_DATE('2025-01-24', 'YYYY-MM-DD'), 110.00, 'Credit Card'),
(5, TO_DATE('2025-01-25', 'YYYY-MM-DD'), 60.00, 'Debit Card'),
(6, TO_DATE('2025-01-25', 'YYYY-MM-DD'), 120.00, 'Cash'),
(7, TO_DATE('2025-01-25', 'YYYY-MM-DD'), 65.00, 'Credit Card'),
(8, TO_DATE('2025-01-25', 'YYYY-MM-DD'), 130.00, 'Debit Card'),
(9, TO_DATE('2025-01-25', 'YYYY-MM-DD'), 70.00, 'Cash'),
(10, TO_DATE('2025-01-25', 'YYYY-MM-DD'), 140.00, 'Credit Card');
```

```
INSERT INTO Snacks (Snack_Name, Snack_Category, Snack_Price, Dep_ID)
VALUES
('Popcorn', 'Snack', 10.00, 3),
('Nachos', 'Snack', 12.50, 3),
('Coke', 'Beverage', 5.00, 3),
('Ice Cream', 'Dessert', 8.00, 3),
('Water', 'Beverage', 3.00, 3),
('Hot Dog', 'Snack', 7.00, 3);
```

Write SQL Queries:

```
-- Basic CRUD Operations (delete)
DELETE FROM Customers WHERE Customer_ID = 1;

-- Basic CRUD Operations (update)
UPDATE Customers
SET Email = 'BahaaAbuSalameh@hotmail.com'
WHERE CustomerID = 1;

--SQL queries that demonstrate the functionality (Basic CRUD Operations(read))

--1 Retrieve the details of the customer with CustomerID = 2.

select * from  Customer where Customer_ID =2;

--2 Calculate the total number of reservations made.
SELECT COUNT(*) AS Total_Reservations
FROM Reservation;

--3 Calculate the average price of all tickets.

SELECT AVG(Price) AS Average_Ticket_Price
FROM Ticket;

--4 Find the maximum payment amount on the date 2025-01-24.

SELECT max(amount) as maximumm_at_date
from Payments
where Payment_Date =TO_DATE('2025-01-24', 'YYYY-MM-DD');
```

```
--5 Calculate the total sales for tickets of type Regular.

SELECT MIN(Price) AS Minimum_Price
FROM Ticket
WHERE Ticket_Type = 'Regular';

--6 Calculate the total sales for snacks in the "snack" category.
SELECT SUM(Snack_Price) AS Total_sncat_Price
FROM Snacks
WHERE Snack_Category = 'snack';

--7  Calculate the total sales for all snacks that contain "Soda" in their name.
SELECT SUM(Snack_Price) AS Total_Sales_Soda
FROM Snacks
WHERE Snack_Name LIKE '%Hot Dog%';

--8  Join to retrieve employee details with the department name
SELECT e.Employee_Name, e.Salary, e.Employee_DOB, d.Department_Name
FROM Employee e
JOIN Department d ON e.Dep_ID = d.Dep_ID;

--9 Join to retrieve a list of snacks with their department names

SELECT s.Snack_Name, s.Snack_Category, s.Snack_Price, d.Department_Name
FROM Snacks s
JOIN Department d ON s.Dep_ID = d.Dep_ID;

--10 Join to retrieve ticket details including the showtime and seat number

SELECT t.Ticket_ID, s.Show_Date, st.Row_num, st.Seat_Number, t.Price
FROM Ticket t
```

```
JOIN Showtimes s ON t.Showtime_ID = s.Showtime_ID
JOIN Seat st ON t.Seat_ID = st.Seat_ID;

--11 Join to retrieve all customers who have made reservations along with the reservation type

SELECT c.Customer_Name, r.Reservation_Date, rt.Reservation_Type_Name
FROM Reservation r
JOIN Customer c ON r.Customer_ID = c.Customer_ID
JOIN Reservation_Type rt ON r.Reservation_Type_ID = rt.Reservation_Type_ID;

--12  Join to retrieve showtimes with hall names and movie details

SELECT s.Show_Date, h.Hall_Name, m.Movie_Name, m.Movie_Type
FROM Showtimes s
JOIN Hall h ON s.Hall_ID = h.Hall_ID
JOIN Movies m ON s.Movie_ID = m.Movie_ID;

--13  Get all movies of type action
SELECT m.Movie_Name, m.Movie_Type, m.Release_Year
FROM Movies m
WHERE m.Movie_Type = 'Action';

--14 Get tickets with a price greater than $10

SELECT t.Ticket_ID, t.Price, s.Show_Date
FROM Ticket t
JOIN Showtimes s ON t.Showtime_ID = s.Showtime_ID
WHERE t.Price > 10;
```

```
--15 Sort employees by salary in ascending order
SELECT e.Employee_Name, e.Salary
FROM Employee e
ORDER BY e.Salary ASC;

--16 Get reservations for a specific customer, sorted by reservation date
SELECT Reservation_ID, Reservation_Date
FROM Reservation
WHERE Customer_ID = (SELECT Customer_ID FROM Customer WHERE
Customer_Name = 'Basel Salah')
ORDER BY Reservation_Date DESC;

--17 Get all customers who have made a reservation on '2025-01-24'

SELECT Customer_Name, Email
FROM Customer
WHERE Customer_ID IN (
   SELECT Customer_ID
   FROM Reservation
   WHERE Reservation_Date = TO_DATE('2025-01-24', 'YYYY-MM-DD')
);

--18 get moive name , rating for movies who has rating grater than 8

SELECT Movie_Name, Rating
FROM Movies
WHERE Rating > 8;

--19  Get the total number of reservations made by each customer

SELECT Customer_ID, COUNT(Reservation_ID) AS Total_Reservations
```

```
FROM Reservation
GROUP BY Customer_ID;

--20  Get the first 5 employees sorted by their hire date (ascending)

SELECT Employee_Name, Hire_Date
FROM Employee
ORDER BY Hire_Date ASC
FETCH FIRST 5 ROWS ONLY;
```